Evaluating Matrix Factorization Robustness in Highly Sparse Movie Recommendation Datasets

Gabriel Guillen
dept. of Statistics & Data Science
University of Central Florida
Orlando, United States of America
ga013701@ucf.edu

***Summary*—This study evaluates three matrix factorization models—Funk Singular Value Decomposition (Funk SVD), Latent Factor Models (LFM-SGD), and Sparse Coding—for movie recommendation. Models were validated using RMSE and other regression-based metrics to assess prediction accuracy, alongside ranking metrics to evaluate recommendation quality. Experimental results indicate that Sparse Coding consistently outperforms the alternative techniques across all metrics, demonstrating superior showing it is better at predicting ratings and works more reliably on new, unseen data.**

I. Introduction

Matrix factorization is essential for mitigating missing data effects in recommendation systems. This study compares three models: **Funk Singular Value Decomposition (Funk SVD)**, **Latent Factor Models (LFM-SGD)**, and **Sparse Coding**.

The primary objective is to predict user ratings despite the extreme sparsity of interactions. We leverage dimensionality reduction to map users and items into a shared latent space, and handling the problem of missing information. Performance is evaluated via **RMSE**, **MAE**, **Precision**, **Recall**, and **NDCG**.

While SVD and LFM-SGD are benchmarks, we hypothesize that **Sparse Coding** is better suited for sparse data. By using an overcomplete dictionary and L_1 regularization, it Focuses on the most important data and ignores errors, capturing subtle preferences that traditional global-variance methods may overlook. While traditional methods provide a baseline, this study focuses on the comparative resilience of Sparse Coding when dealing with a dataset where 93.7% of the ratings are missing.

A. Dataset Structure

This study utilizes the **MovieLens 100K** [1] dataset, which consists of 100,000 ratings ($R \in \{1, 2, 3, 4, 5\}$) provided by 943 users across 1,682 movies. The dataset exhibits a sparsity common in recommendation tasks, with a density of 6.304%.

The data is partitioned into training and testing subsets to facilitate model validation. Each observation is structured in a four-column format: `'user item rating timestamp'`.

Auditing of the entries confirmed a clean feature space, allowing for direct mapping of user-item interactions into the latent factor models.

II. Methods

A. Model Preparation

The models were trained on a sparse **User-Item interaction matrix** [1] of 943 users (U) and 1,682 movies (I). Using pre-partitioned subsets, the models learned latent factors to predict ratings that were kept hidden during training.

B. Performance Evaluation

To identify which model best captures subtle preferences in sparse data, performance was measured using several predictive and ranking metrics. Alongside **Mean Absolute Error (MAE)** [6], the primary error metric was **Root Mean Square Error (RMSE)** [1]:

$$RMSE = \sqrt{\frac{\sum_{u \in U, i \in I} (R_{ui} - R'_{ui})^2}{Number\ of\ Ratings}} \tag{1}$$

Where R_{ui} and R'_{ui} represent the actual and predicted ratings for user u and item i, respectively.

We complement RMSE with **Normalized Discounted Cumulative Gain (NDCG)** [7] to evaluate ranking quality, using logarithmic decay to reward models that surface relevant items.

To further assess relevance, **Precision** [6] and **Recall** [6] were calculated using a threshold of 3, where ratings ≥ 3 are classified as "good" and others as irrelevant.

This multi-metric approach provides a robust benchmark framework.

C. Matrix Factorization Models Used

1) **Funk Singular Value Decomposition (Funk SVD)** [3]: A latent factor benchmark that factorizes the user-item matrix by optimizing solely over observed ratings via SGD. Configured with 50 latent factors ($k = 50$), a learning rate ($\eta = 0.005$), and regularization ($\lambda = 0.02$) over 50 epochs, it extracts latent features without the bias of dense imputation.
2) **Latent Factor Models (LFM-SGD)** [4]: A model utilizing Stochastic Gradient Descent to learn user/item embeddings ($k = 20$ latent factors, 20 iterations). By optimizing only over observed entries, it scales efficiently to sparse matrices and captures nuanced patterns without requiring data imputation.

3) **Sparse Coding** [5]: A generative framework that represents data as a sparse linear combination of atoms from an overcomplete dictionary. Utilizing Elastic Net regularization (with $\lambda = 0.01, \alpha = 0.01$), it isolates salient features by activating only the most relevant basis functions, effectively filtering noise.

III. Experimental Results and Metric Divergence

The experimental results as shown in Figure 1 indicate a a big difference between a model's ability to rank movies correctly and its ability to guess the exact rating. While **Sparse Coding** achieves a near-optimal **NDCG of 0.99**, its **Norm_Accuracy** remains low at **0.13**.

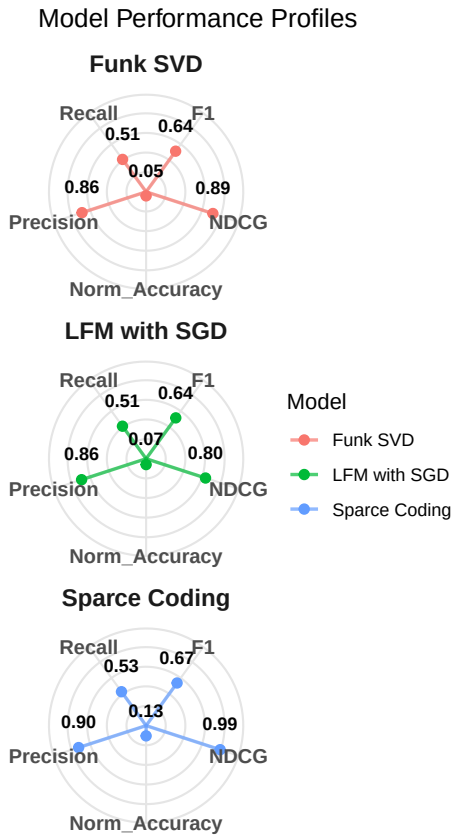

Fig. 1. Models Profile By Measuring Metrics

This divergence suggests that the model effectively captures relative preference hierarchies (ranking) despite high absolute error in point-wise rating prediction ($Norm\_Accuracy = 1 - RMSE_{test}$).

Similarly, **LFM-SGD** and **Funk SVD** demonstrate robust ranking capabilities with **NDCG** values of **0.80** and **0.89** respectively, yet yield negligible **Norm_Accuracy** scores (≤ 0.07).

This gap shows that while models understand which movies a user likes more than others, they struggle to hit adequate **RMSE**.

Consequently, these models are highly effective for recommendation retrieval where item ordering is prioritized over exact rating accuracy.

IV. Discussion

A. *Comparative Model Performance*

The performance analysis in Figure 2 reveals a consistent trend in generalization, with test error remaining comparable to training error across all models.

This stability suggests that underlying patterns were captured without significant overfitting; however, the **persistent magnitude of the RMSE** indicates that the models have reached a predictive floor.

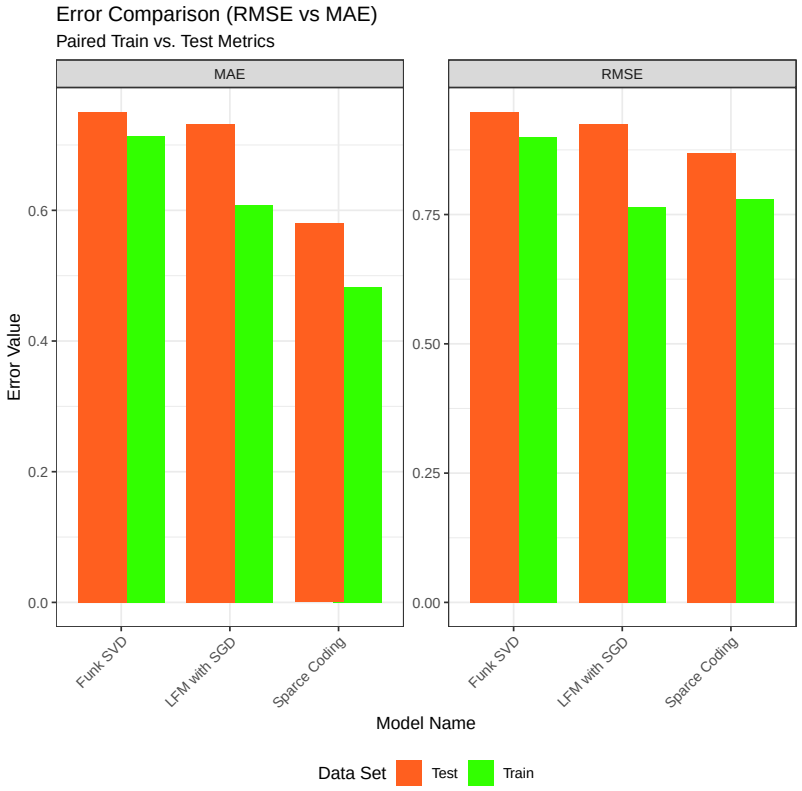

Fig. 2. Error Comparison by Model

Sparse Coding demonstrates the highest efficiency, achieving the lowest MAE and RMSE values. Its superior training and test results indicate it is the most capable model for mapping user features to ratings.

In contrast, **LFM-SGD** and textbfFunk SVD exhibit higher baseline error rates. While their training and test metrics are

closely aligned, their higher values indicate less precision in predicting exact ratings compared to Sparse Coding.

B. *Model Generalization*

Observing the slope graph in Figure 3 **Sparse Coding** achieved the highest performance, maintaining a stable NDCG of approximately 0.99 across both training and testing datasets. This minimal variance indicates exceptional generalization and high accuracy on unseen data.

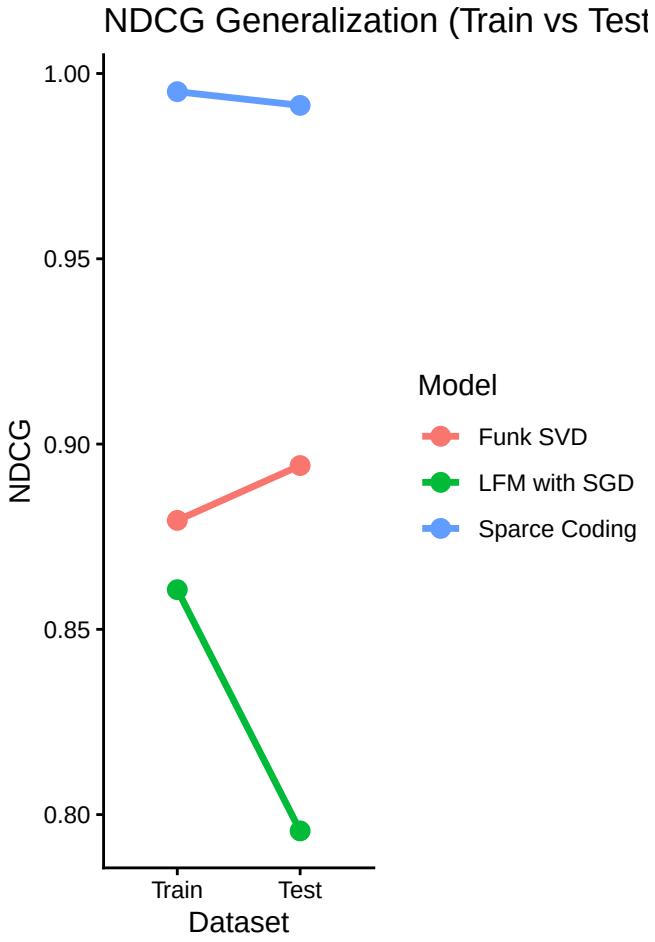

Fig. 3. NDCG by Model

Funk SVD exhibited a unique upward trend, with its NDCG rising from 0.88 during training to 0.89 in the testing phase. This slight improvement suggests the model is highly robust and effectively avoids overfitting.

LFM-SGD proved to be the weakest model. Its NDCG dropped sharply from 0.86 on the training set to below 0.80 on the test set.

This significant decline represents the largest generalization gap in the analysis, indicating substantial overfitting.

C. *Evaluating Model Recommendation Efficacy*

The visualization highlights shown in Figure 4 the trade-off between **Precision (the quality of recommendations)** and **Recall (the breadth of recommendations)**. **Sparse Coding** emerges as the superior model, occupying the top-right position with a **Precision** ≈ 0.90 and **Recall** ≈ 0.54, effectively maximizing relevance while minimizing noise.

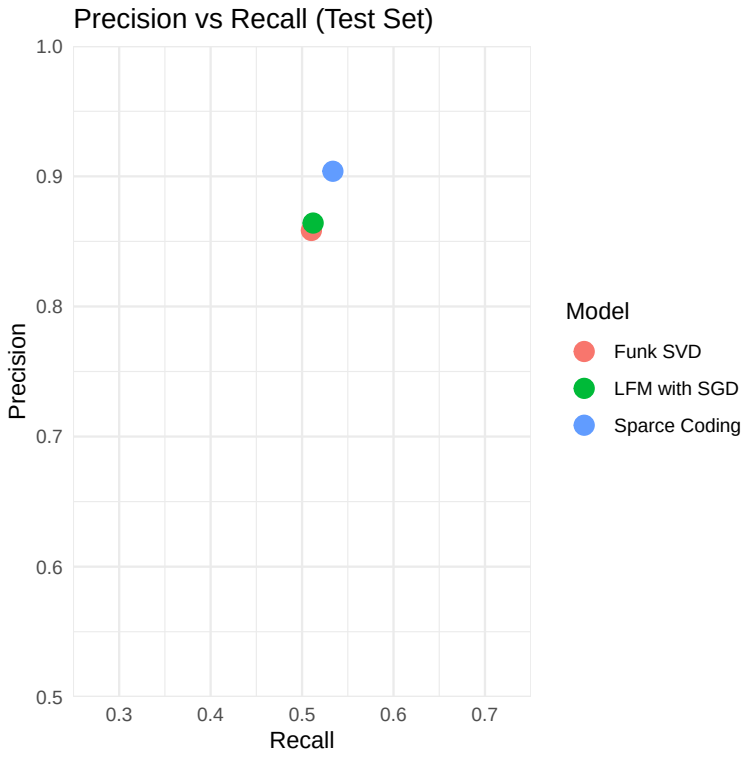

Fig. 4. Precision-Recall Comparison Among Models

LFM-SGD occupies the middle tier, yielding a **Precision** ≈ 0.86 and **Recall** ≈ 0.51. Finally, **Funk SVD** is the least performant model, sitting at the base of the cluster with a **Precision** of 0.85 and the lowest **Recall** at 0.50, capturing the smallest portion of relevant items.

V. Conclusion

This study evaluated three matrix factorization models—**Funk SVD**, **LFM-SGD**, and **Sparse Coding**—on the MovieLens 100K dataset.

Despite the 93.7% sparsity and high dimensionality, **Sparse Coding** emerged as the most robust framework, achieving superior precision and generalization.

A critical finding is the divergence between error-based and ranking-based metrics. While **RMSE** measures point-wise prediction magnitude, it proved insufficient as a sole indicator of quality. Even as models reached a "predictive floor" in absolute accuracy, their ranking proficiency remained high.

The inclusion of **NDCG** was essential for this evaluation. While **RMSE** quantifies distance from actual values, **NDCG** rewards the correct ordering of recommendation lists.

Sparse Coding achieved a near-perfect **NDCG** of 0.99, demonstrating that it effectively find and recommend the right movies even when absolute predictions contain noise.

For real-world systems, where "top-N" accuracy is paramount, ranking proficiency is a more vital indicator of user satisfaction than **RMSE**.

The superior performance of **Sparse Coding** across **Precision-Recall**, **NDCG**, and **RMSE**—relative to both **Funk SVD** and **LFM-SGD**—confirms its enhanced ability to isolate salient features within sparse matrices.

Because **Sparse Coding** achieves high NDCG and Precision, it ensures that the 'highest predicted ratings' generated by the model truly correspond to the items the user is most likely to enjoy.

REFERENCES

[1] Harper, F. M., and Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4), 1–19. https://doi.org/10.1145/2827872

[2] Papers with Code. (2026). Collaborative Filtering on MovieLens 100K. https://paperswithcode.com/sota/collaborative-filtering-on-movielens-100k

[3] S. Funk. Netflix Update: Try This at Home. http://sifter.org/~simon/journal/20061211.html, 2006. Accessed: May 20, 2024.

[4] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.

[5] Olshausen, B. A., and Field, D. J. (1996). Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583), 607–609.

[6] James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R* (2nd ed.). Springer Texts in Statistics. New York: Springer.

[7] Järvelin, K., and Kekäläinen, J. (2002). *Cumulated gain-based evaluation of IR techniques*. ACM Transactions on Information Systems (TOIS), 20(4), 422–446. https://doi.org/10.1145/582415.582418

APPENDIX

APPENDIX A: SUPPLEMENTARY MATERIALS AND SOFTWARE USAGE

Source Code Repository

The source code, data, and detailed implementation scripts for this project are available on GitHub (URL: https://github.com/gaguillen4384-dev/Data-Science/tree/main/DataMining_2/Movie_Recommender_Model_Analysis).

Software Usage Acknowledgment

1) The text and documentation were edited and refined using **Google Gemini**.
2) **R** (version 4.x.x) was used for model computation and graphics development.
3) The following **R** libraries were essential to the computational workflow:
   - **recosystem**: Utilized for Latent Factor Modeling via Stochastic Gradient Descent (LFM-SGD).
   - **glmnet**: Applied for Sparse Coding through regularized generalized linear models.
   - **Matrix**: Used to manage and manipulate sparse matrix structures required for high-dimensional computations.
   - **jsonlite**: Employed for the robust parsing and processing of JSON-formatted data.
   - **dplyr** & **tidyr**: Used for data manipulation, cleaning, and reshaping into tidy formats.
   - **ggplot2**: Used for the development of all statistical graphics and data visualizations.